Distributed Multi-Model Analytics for E-commerce Data

AUCA Big Data Analytics - Final Project Technical Report

SEBAGANWA SHAMI DELICE | Student ID: 101371 | 11 November 2026

1. Overview and system architecture

The project implements a multi-model analytics platform for a synthetic e-commerce business. The provided `dataset_generator.py` was run at 50% of the assignment scale (**1,000,000 sessions, 250,000 transactions**, 10,000 users, 5,000 products, 25 categories, 90 days; 1.9 GB of JSON). Each technology is used for the job it is best at: **MongoDB** stores the rich, nested business entities (catalog, customers, orders) and answers entity-centric aggregations; **HBase** stores the high-volume, append-only, sparse click-stream as a time-series keyed for fast per-user and per-product range scans; **Apache Spark** cleans the raw files into a Parquet lake, runs the heavy batch analytics (market-basket, cohorts, Spark SQL) and acts as the integration engine that joins data pulled from both databases.

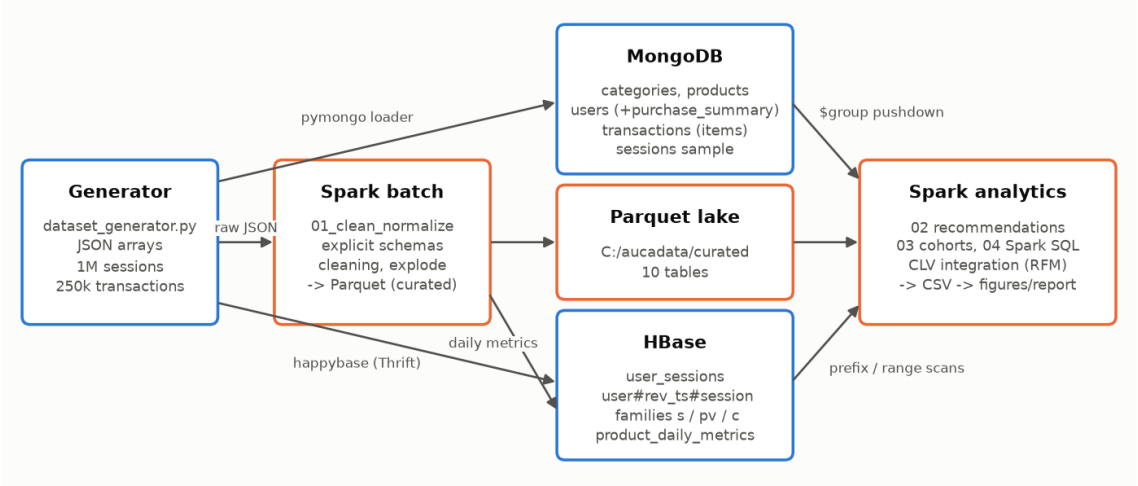


Figure 1 - Architecture: generator -> Spark cleaning -> MongoDB / HBase / Parquet -> Spark analytics -> figures and report.

Environment. Windows 11 laptop (20 cores, 32 GB RAM); PySpark 3.5.3 in local[*] mode on Java 8; MongoDB 7.0.45 (host port 27018) and HBase 2.1.3 (standalone, Thrift on :9090) in Docker via `docker-compose.yml`. Large data lives in `C:/aucadata` (outside the source repository). The whole pipeline is reproducible with `run_all.ps1` (see README).

2. Data modelling decisions (Part 1)

2.1 MongoDB document model

The document model is designed around the read patterns: an entity and everything that is always read with it lives in one document; unbounded or high-volume relationships are referenced.

Collection	Shape	Justification
categories	{_id, name, subcategories:[{subcategory_id, name, profit_margin}], avg_profit_margin}	The hierarchy is small (3-5 subcategories) and always read with its parent -> embedded; avg margin pre-computed for profit analytics.
products	{_id, name, category:{category_id, name}, base_price, current_stock, is_active, price_history:[{price, date}], creation_date}	Bounded price history (1-3 points) embedded; extended reference to the category (id + name) avoids a $lookup for listing pages. Indexed on category and (is_active, stock).
users	{_id, geo:{city,state,country}, registration_date, last_active, purchase_summary:{total_orders, total_spent, avg_order_value, first/last_purchase, days_since_last_purchase, top_payment_method}, preferences:{favourite_category_id/name, favourite_category_spent, categories_bought}}	Computed pattern: the purchase summary and preferences are materialised by aggregations ending in $merge, so a profile page, CRM lookup or personalisation is a single _id read instead of an aggregation over ~25 orders.

Collection	Shape	Justification
transactions	{_id, session_id, user_id, timestamp, items:[{product_id, category_id, quantity, unit_price, subtotal}], subtotal, discount, total, payment_method, status}	Line items are owned by the order and bounded -> embedded. category_id is denormalised into each item so revenue-by-category needs no join. Indexes: (user_id, timestamp), timestamp, items.product_id, session_id (sparse).
sessions (sample)	Full journey document: device, geo, referrer, page_views[], cart[], conversion_status	Only a 100k sample: ideal for ad-hoc journey analysis of a single session; the full click-stream belongs in HBase.

Aggregation pipelines (`mongodb/aggregations.py`): (1) *revenue analytics* - $unwind items, $group by category and month, $lookup to the category hierarchy for the margin, estimated gross profit and a $setWindowFields $rank of categories inside each month; (2) *user segmentation* by purchasing frequency (the brief's other example, age group, is not possible: the generated user records contain only user_id, city/state/country, registration_date and last_active, with no age or birth date) - $group per user (orders, spend, recency, favourite payment via $sortArray), $bucket by purchase frequency, and the same stages ending in $merge to write `users.purchase_summary` (a second $merge stores each user's favourite category in `users.preferences`); (3) *product popularity* - most-viewed products from session page views joined with a correlated $lookup to their sales, plus top sellers. The sessions sample also supports whole-journey questions: one document gives entry page, referrer, device, every page view and the cart (mongodb/shell_queries.js). Excerpt of pipeline (1):

```
db.transactions.aggregate([
  {$unwind: "$items"},
  {$group: {_id: {category_id: "$items.category_id",
                  month: {$dateToString: {format: "%Y-%m", date: "$timestamp"}}},
            gross_revenue: {$sum: "$items.subtotal"}, units_sold: {$sum: "$items.quantity"},
            orders: {$addToSet: "$_id"}}},
  {$lookup: {from: "categories", localField: "_id.category_id", foreignField: "_id", as: "category"}},
  {$unwind: "$category"},
  {$project: {..., est_gross_profit: {$multiply: ["$gross_revenue", "$category.avg_profit_margin"]}}},
  {$setWindowFields: {partitionBy: "$month", sortBy: {gross_revenue: -1},
                      output: {rank_in_month: {$rank: {}}}}])
```

2.2 HBase wide-column model

Table	Row key	Column families	Access pattern served
user_sessions	user_id # (9999999999 - start_epoch) # session_id	s: session summary (15 cols) | pv: one column per page view, qualifier 000..015, value ts|type|product|category|secs | c: one column per cart product, value qty|price	Latest N sessions of a user = prefix scan with LIMIT (newest first thanks to the reverse timestamp); sessions in [t1,t2] = bounded range scan; one session's click-stream = single-row get on 'pv'.
product_daily_metrics	product_id # yyyyMMdd	m: views, view_seconds, unique_sessions, cart_adds, cart_units, orders, units_sold, revenue	A product's performance over any date range = one contiguous range scan -> view-to-cart and view-to-purchase conversion per period.

Design choices: the time dimension is modelled in the **row key**, not in cell versions (VERSIONS=1); the variable-width, sparse page-view list becomes dynamic column qualifiers (4-16 per row, empty cells cost nothing); families are split by access frequency so summary scans never read page-view blocks from disk; tables are **pre-split** on user / product ranges, with ROW bloom filters, FAST_DIFF block encoding and GZ compression (`hbase/schema.hbase`). A pure user_id prefix could hot-spot a region if one key range were very active; at larger scale a 1-byte salt (hash(user_id) mod N) would be prepended while keeping per-user scans contiguous. Schema and sample queries (HBase shell):

```
create 'user_sessions',
  {NAME => 's', VERSIONS => 1, BLOOMFILTER => 'ROW', DATA_BLOCK_ENCODING => 'FAST_DIFF', COMPRESSION => 'GZ'},
  {NAME => 'pv', ...}, {NAME => 'c', ...},
  SPLITS => ['user_002000', 'user_004000', 'user_006000', 'user_008000']
# latest 5 sessions of a user (newest first thanks to the reverse timestamp)
scan 'user_sessions', {ROWPREFIXFILTER => 'user_000042#', COLUMNS => ['s'], LIMIT => 5}
# one week of one product
scan 'product_daily_metrics', {STARTROW => 'prod_03462#20261101', STOPROW => 'prod_03462#20261108'}
```

2.3 Why each dataset lives where it does

Data	Store	Reason
Products, categories, users, transactions	MongoDB	Rich nested entities, moderate volume, many different ad-hoc queries and secondary indexes; aggregation framework for business analytics; flexible schema (e.g. missing subcategory_id).
Session click-stream (10M page views)	HBase	Append-only, time-ordered, very high volume and sparse; queried almost exclusively by key range (user + time, product + day); scales linearly by adding region servers.
Daily product metrics	HBase	Classic time-series counter table (could be updated with atomic increments from a stream).
Curated copies of everything	Parquet (Spark)	Columnar, compressed, splittable input for full-scan batch analytics that neither database is optimised for.

3. Spark processing pipelines (Part 2)

01_clean_normalize.py. Reads the raw JSON arrays with *explicit schemas* (inference would cost an extra pass over 1.9 GB and would turn `cart_contents`, whose keys are product ids, into a struct with thousands of fields - it is declared as a MapType). Cleaning: drop rows without keys, de-duplicate on ids (one duplicate session id was found - the generator truncates UUIDs to 10 hex chars), trim/lower-case categorical strings, default missing device/referrer to 'unknown', clamp negative stock and durations, parse ISO timestamps, recompute line totals and flag orders where subtotal - discount != total (0 found). Nested arrays are exploded into relational tables (page_views, cart_items, transaction_items, product_price_history, subcategories) with `posexplode` to keep event order; per-session features (page-type counts, cart value) are computed with higher-order functions (filter/aggregate) without exploding. Output: 10 snappy-Parquet tables, page_views partitioned by date. The JSON arrays need multiLine=true, which makes each file a single task; converting once to Parquet removes that bottleneck for every later job. Runtime: 4.1 min.

02_recommendations.py (required batch analysis). Market-basket rules for both "users who bought X also bought Y" (baskets = transactions) and "products frequently viewed together" (baskets = sessions). A self-join on the basket id with `a.product_id < b.product_id` counts each unordered pair once (halving the shuffle); pairs are filtered at a minimum count of 3, expanded to both directions and enriched with item frequencies via broadcast joins to compute support, confidence and lift. Both signals are blended (0.7 x co-bought + 0.3 x co-viewed) and a `row_number()` window keeps a top-5 list per product. **03_cohort_analysis.py** (additional) groups users by registration month and measures active buyers, retention and revenue per cohort user per purchase month.

04_spark_sql_analytics.py. All curated tables are registered as temp views and queried with SQL: A1 daily revenue with a 7-day moving average and LAG growth; A2 category scorecard with CTEs, revenue share (SUM() OVER ()), RANK and margin-based profit; A3 conversion funnel with conditional aggregation and ROLLUP(device, referrer); A4 a FULL OUTER JOIN of views, cart adds and sales per product; A5 weekday x hour traffic. Section B registers `mongo_users` (documents read with pymongo) and `hbase_sessions` (rows read with happybase prefix scans) as views and joins them with Parquet transactions in a single SQL statement - the connector-free way of querying data that lives in the two stores. Example (A3, conversion funnel):

```
SELECT device_type, referrer, COUNT(*) AS sessions,
       SUM(CASE WHEN num_product_detail_views > 0 THEN 1 ELSE 0 END)    AS viewed_product,
       SUM(CASE WHEN num_cart_items > 0 THEN 1 ELSE 0 END)              AS added_to_cart,
       SUM(CASE WHEN num_checkout_views > 0 THEN 1 ELSE 0 END)          AS reached_checkout,
       SUM(CASE WHEN conversion_status = 'converted' THEN 1 ELSE 0 END) AS converted
FROM sessions GROUP BY ROLLUP(device_type, referrer)
```

Optimisations used: explicit schemas; Parquet + column pruning; date partitioning of the largest table; broadcast joins for dimensions; caching of re-used inputs; adaptive query execution; 64 shuffle partitions sized for 20 cores; MongoDB aggregations pushed down so only per-user summaries are transferred; HBase scans restricted to one family.

Robustness: explicit schemas turn malformed values into nulls instead of failing the job; rows without keys are dropped and ids de-duplicated; totals are re-validated (total_is_consistent flag); MongoDB loads use unordered bulk inserts that report and skip rejected documents (BulkWriteError) instead of aborting; HBase writes are batched; every step is idempotent (Parquet overwrite, collection drop/reload, table re-create) so the pipeline can be re-run safely; connection timeouts make an unavailable database fail fast with a clear error; the generator keeps its MAX_ITERATIONS fail-safe.

4. Integrated analytics workflow (Part 3): Customer Lifetime Value

Business question: which customers will generate the most gross profit over the next 12 months, how does browsing engagement relate to that value, and which segments should marketing prioritise?

Sources and steps (`integration/clv_integration.py`): (1) MongoDB *users* -> profile and geography; (2) MongoDB *transactions* -> two aggregations executed inside MongoDB (orders, revenue, first/last purchase per user; and revenue per user x category joined via $lookup to category margins) so only ~10k rows leave the database; (3) HBase *user_sessions* -> a full scan restricted to six columns of family 's' gives sessions, duration, pages, conversion and abandoned-cart value per user; (4) Spark joins the three DataFrames, computes RFM quintiles with `ntile(5)` windows, assigns rule-based segments and the CLV model:

```
CLV_12m = (orders / observed_days) x 365 x exp(-recency_days / 30)
         x avg_order_value x margin-weighted gross margin
```

Spark is the right place for step 4 because it can join heterogeneous sources in memory, apply window functions over the whole customer base and scale the same code to a cluster; the databases do what they are indexed for (MongoDB: group by user; HBase: key-range scans). **Performance (measured):** the two MongoDB aggregations returned 10,000 per-user summaries in 7.8 s; the HBase scan of 300,000 session rows (6 columns of family 's' only) took 8.8 s; the whole job including Spark start-up took 91.8 s. Point look-ups are interactive: HBase 'latest sessions of a user' 6.5 ms, product date-range scan 17.0 ms; the MongoDB per-user order query uses the (user_id, timestamp) index and examines only the matching documents (IXSCAN). With millions of customers the HBase side would be read with the HBase-Spark connector (region-parallel scans) instead of a single Thrift client.

The same sources are also joined declaratively in Spark SQL (04_spark_sql_analytics.py, section B):

```
WITH engagement AS (                                          -- rows scanned from HBase
    SELECT user_id, COUNT(*) AS hbase_sessions, AVG(duration_seconds) AS avg_session_s,
           SUM(CASE WHEN conversion_status = 'converted' THEN 1 ELSE 0 END) AS converted_sessions
    FROM hbase_sessions GROUP BY user_id)
SELECT u.user_id, u.total_orders, u.total_spent, e.hbase_sessions, e.converted_sessions
FROM mongo_users u                                             -- documents read from MongoDB
JOIN engagement e ON u.user_id = e.user_id
ORDER BY u.total_spent DESC
```

Segment	Users	Avg CLV 12m	Avg orders	AOV	Recency (d)	HBase sessions	Session conv.
Champions	1,483	$32,151	30.3	$1,057	0.5	31.3	23.1%
Loyal high-value	2,242	$27,018	29.7	$1,009	3.9	31.1	23.2%
Core	1,347	$24,163	25.1	$982	1.3	30.0	20.9%
At-risk big spenders	417	$23,878	24.4	$1,181	6.4	29.4	19.0%
Promising / low-frequency	2,088	$20,429	20.4	$1,019	1.2	28.8	17.0%
Hibernating	2,423	$17,031	21.4	$974	6.9	29.3	18.4%

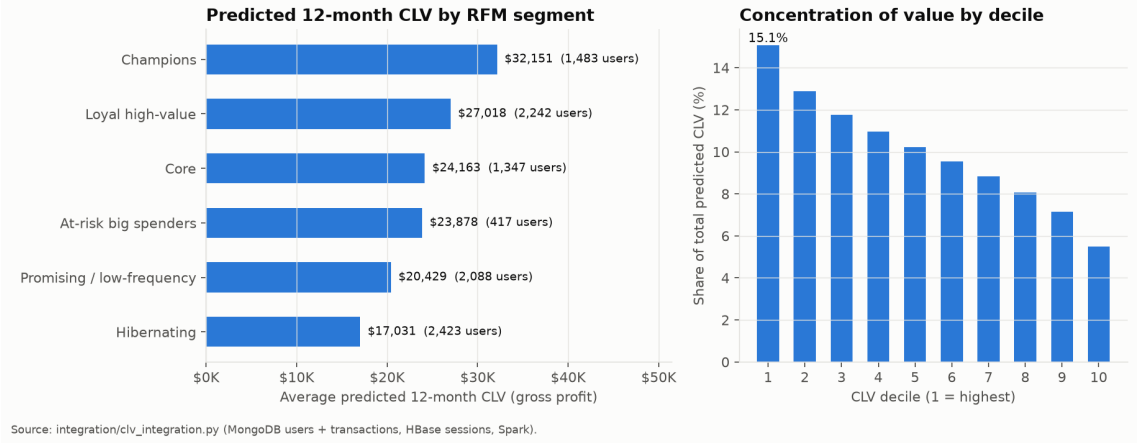


Figure 2 - Predicted 12-month CLV by RFM segment and concentration by decile (MongoDB + HBase + Spark).

5. Key findings and business insights (Part 4)

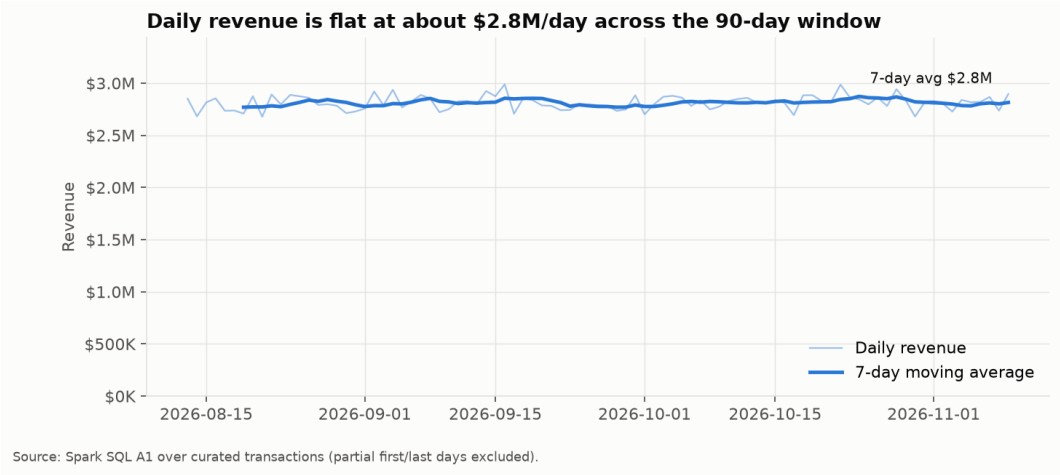


Figure 3 - Daily revenue (Spark SQL A1). Total $253.2M over 90 days, 250,000 orders, average order value $1,013.

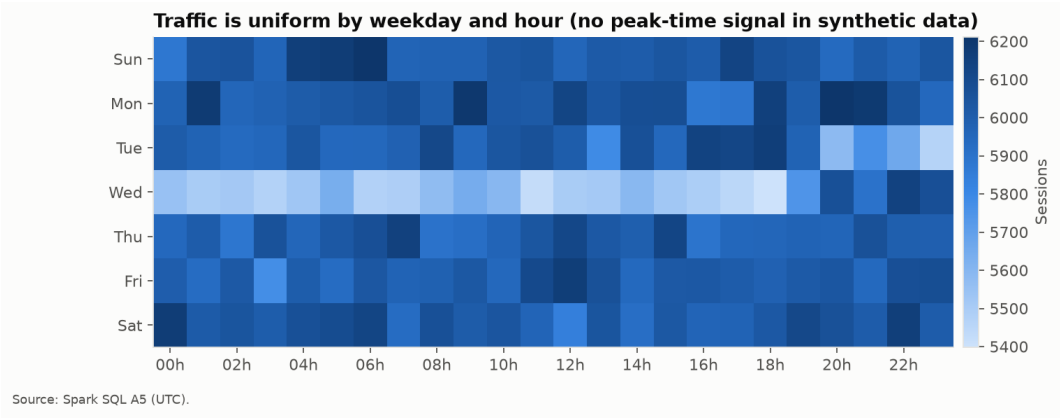


Figure 4 - Sessions by weekday and hour, UTC (Spark SQL A5).

• **Stable demand.** Revenue averages $2.8M/day (std 2.4%) with no trend or weekly seasonality, and Figure 4 shows equally flat weekday x hour traffic. Forecasting can use a simple baseline; capacity planning does not need peak-hour provisioning for this traffic profile.

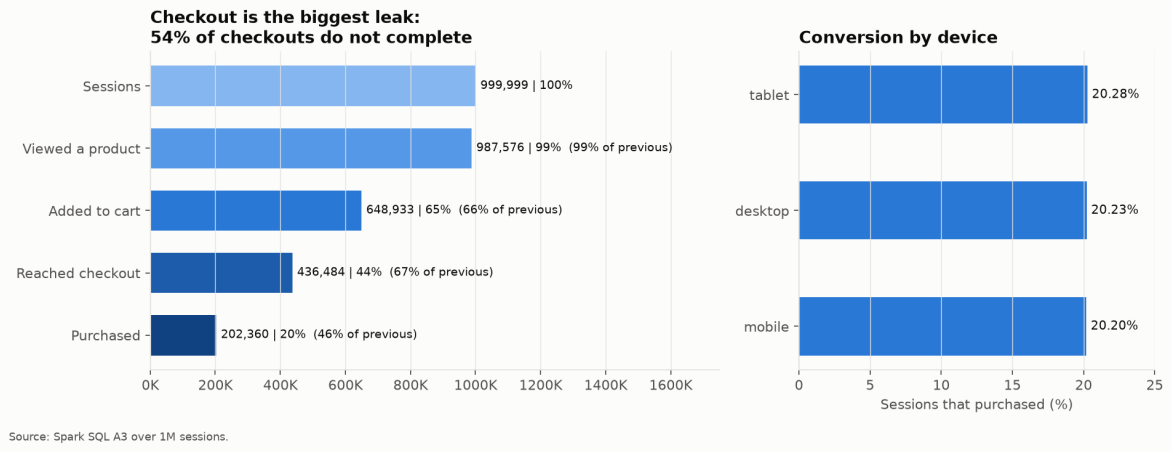


Figure 5 - Session conversion funnel and conversion by device (Spark SQL A3 over 1M sessions).

• **Checkout is the largest leak.** 99% of sessions view a product and 65% add to cart, but only 46% of sessions that reach checkout complete it (54% drop out). Abandoned carts are the cheapest revenue to recover: checkout UX, saved carts and reminder e-mails should be prioritised over acquisition.

• Conversion is identical across devices (20.2-20.3%) and referrers (20.1-20.4%), so channel budget should follow cost per session rather than expected conversion.

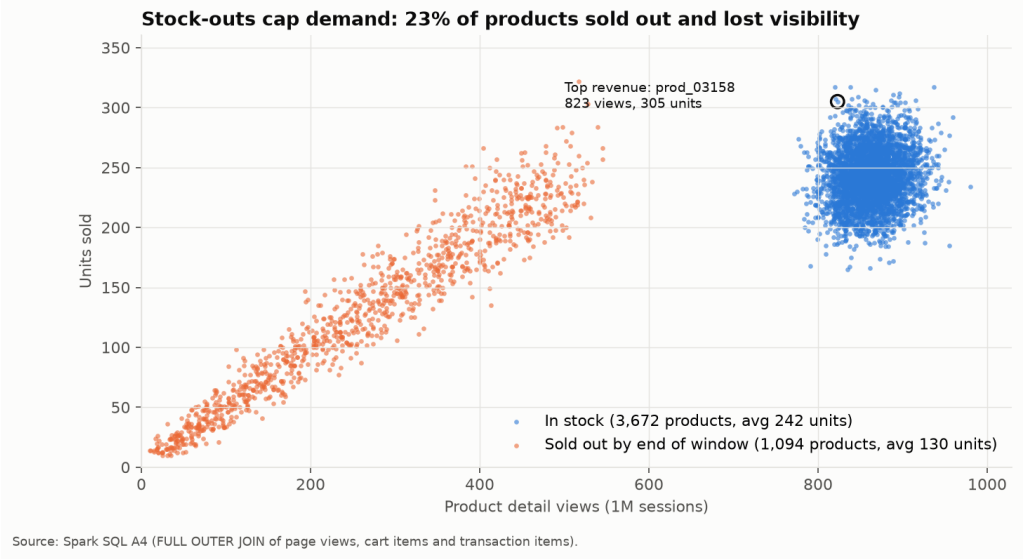


Figure 6 - Product views vs units sold (Spark SQL A4); orange = products with zero stock at the end of the window.

• **Stock-outs are the biggest product-level loss.** 1,094 of 4,766 active products (23%) sold out during the window. Because the storefront stops showing them, they averaged 265 views and 130 units versus 863 views and 242 units for in-stock products. Had they kept selling at the in-stock rate, roughly **$31.5M** of additional revenue was possible (12% of total). Recommendation: reorder alerts driven by the HBase product_daily_metrics table (units_sold trend vs current_stock).

• Within in-stock products, views are the dominant driver of sales (view-to-order 14.1%), so merchandising (search ranking, category placement) directly moves revenue.

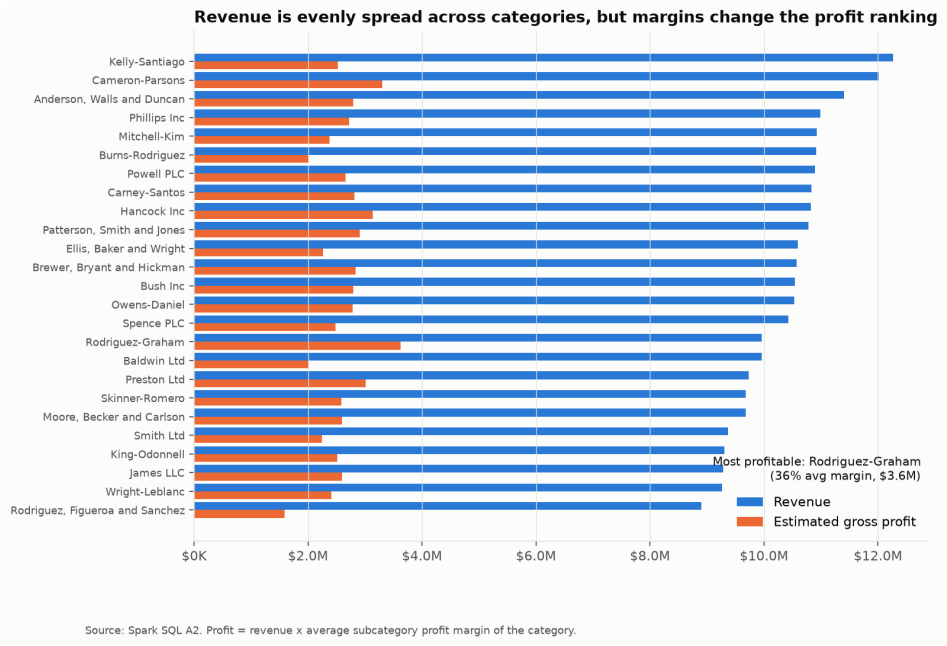


Figure 7 - Revenue and estimated gross profit per category (Spark SQL A2).

• **Revenue is spread evenly but profit is not.** Category revenue share ranges only from 3.4% to 4.7% (leader: Kelly-Santiago, $12.3M), but margins range from 18% to 36%. Rodriguez-Graham is the most profitable category ($3.6M) despite ranking #16 by revenue. Promotions and discounts (currently applied uniformly to 20% of orders) should be steered towards high-margin categories.

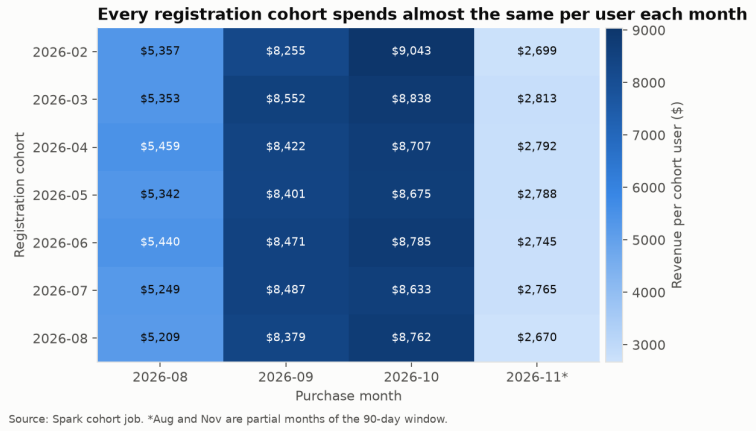


Figure 8 - Revenue per cohort user by purchase month (Spark cohort analysis).

• **No cohort effect.** Older and newer registration cohorts spend almost exactly the same per user each month (October: $8,633-$9,043) and monthly retention is 99.9%+. In real data this would be suspicious; here it reflects the generator assigning sessions and orders to users uniformly at random (see Limitations).

• **Recommendations need minimum support.** Among 250k orders, no product pair was bought together more than 3 times, so the very high lifts of bought-together rules (up to 113) are noise from tiny counts. Co-viewing is denser (pairs up to 9 sessions, median lift ~4), which is why the blended score also uses view co-occurrence; in production a support threshold and a significance test would be applied.

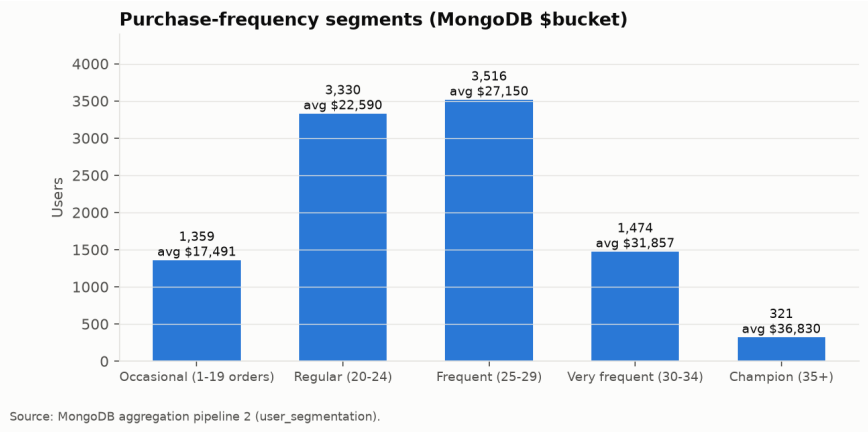


Figure 9 - Purchase-frequency segments and average spending per user (MongoDB $bucket pipeline). Segments use purchase frequency instead of age groups because the dataset has no age field.

• **CLV is driven by purchase frequency, not browsing time.** The top decile holds 15.1% of predicted 12-month CLV versus 5.5% for the bottom decile. CLV correlates with orders (r = 0.78) and order value (r = 0.48); among the HBase engagement metrics only session conversion rate matters (r = 0.24), while session count (r = 0.14) and session duration (r = -0.00) carry almost no signal. Engagement should be measured by conversion, not time on site.

• **Segment actions.** 1,483 Champions are worth $32,151 each (loyalty programme, early access). 'Hibernating' customers left the most value in abandoned carts in the HBase sample ($26.1M), making them the first target for cart-recovery e-mails.

6. Technology selection and scalability

Analytical task	Chosen technology	Why
Customer profile / order look-ups, revenue by category, segmentation	MongoDB aggregation	Indexed, document-local data; results needed interactively; $merge materialises summaries.
Latest sessions of a user, product metrics over a date range	HBase scans	Millisecond key-range scans regardless of table size; sparse wide rows.
Cleaning 1.9 GB JSON, market-basket self-joins, cohorts, funnel over 10M events	Spark (Parquet)	Full scans and shuffles across all data; parallel and scalable to a cluster.
CLV across stores	Spark as integration layer	Joins heterogeneous sources and applies window functions over the whole customer base.

• **MongoDB** scales reads with replica sets and writes/volume with sharding; transactions would be sharded on a hashed user_id (even distribution, user-local queries) and time-range analytics served from a secondary or Atlas Data Lake.

• **HBase** scales horizontally by region splits over more region servers; the pre-split, (salted) row keys avoid write hot-spots, and TTLs on user_sessions would age out old click-stream automatically.

• **Spark** code runs unchanged on YARN/Kubernetes; at full scale (2M sessions) the cleaning job is I/O bound and parallel per file. The main memory risk at full scale is the generator itself, which holds every session in Python memory (~25 GB at 100%), which is why 50% scale was used; a streaming writer (one JSON-lines record at a time) would remove the limit.

• Streaming extension: page-view events could be published to Kafka and processed with Spark Structured Streaming, writing user_sessions rows and incrementing product_daily_metrics counters in HBase in near real time.

7. Methodology

• Funnel stages are measured per session from page-type counts (product_detail, cart content, checkout) and conversion_status.

• Gross profit is estimated as revenue x the mean subcategory profit_margin of the product's category, because the generator does not assign products to subcategories.

• Market-basket metrics follow the standard support / confidence / lift definitions with a minimum pair count of 3.

• Cohorts are defined by registration month; months_since_registration uses months_between on month-truncated dates.

• Customer segmentation uses behaviour, not demographics: purchase-frequency buckets (MongoDB) and RFM quintiles (Spark). The assignment suggests age groups as an example, but dataset_generator.py creates users without any age or date-of-birth field (only geography and registration/activity dates), so an age-based segmentation could not be computed honestly.

• CLV uses a transparent heuristic (purchase rate x expected lifetime with exponential recency decay x AOV x margin) instead of a fitted BG/NBD model, which would be the next step with real, non-uniform data.

8. Limitations and future work

• **Synthetic data:** users, products and timestamps are drawn uniformly at random, so there is no seasonality, no cohort effect, almost no real product affinity and identical channel/device behaviour. Several 'flat' findings above are properties of the generator, not of customer behaviour; the pipeline itself is what transfers to real data.

• Generator quirks found during cleaning: products have no subcategory_id; session ids are truncated UUIDs (1 duplicate in 1M); users have no age (see Methodology); 50% of orders (124,092) are not linked to any session, which caps funnel-to-order attribution.

• Scale: 50% of the specified volume because of generator memory; HBase holds a 300k-session subset; MongoDB holds a 100k-session journey sample.

• **Future work:** MongoDB Spark / HBase-Spark connectors instead of client libraries (partition-parallel reads), Structured Streaming ingestion, ALS recommendations (MLlib), probabilistic CLV (BG/NBD) and a dashboard.

Appendix A - Repository layout and how to reproduce

data_generation/ (generator), common/config.py (paths, Spark session), spark/01-04 (batch + SQL), mongodb/ (loader, pipelines), hbase/ (schema.hbase, loader, queries), integration/clv_integration.py, visualization/make_charts.py, report/build_report.py, output/results (CSV), output/figures (PNG), docker-compose.yml, run_all.ps1, README.md. Run `.\run_all.ps1` after creating the virtual environment and starting Docker Desktop.

HBase sample query output: product daily funnel with 91 daily rows retrieved by one range scan (output/results/hbase_product_funnel_sample.csv).

Appendix B - Execution evidence (screenshots)

Console output captured while running each step (tools/capture.py); full logs are in output/logs/.

Windows PowerShell - 00_generator_log

```
PS dataanalytics> Get-Content C:/aucadata/generate.log
Initializing dataset generation...
Generated 25 categories
Generated 5000 products
Generated 10000 users
Generating sessions and transactions...
Progress: 200,000/1,000,000 sessions, 80,516/250,000 transactions (iteration 200,000)
Progress: 400,000/1,000,000 sessions, 160,805/250,000 transactions (iteration 400,000)
Progress: 600,000/1,000,000 sessions, 240,594/250,000 transactions (iteration 600,000)
Progress: 800,000/1,000,000 sessions, 250,000/250,000 transactions (iteration 800,000)
Progress: 1,000,000/1,000,000 sessions, 250,000/250,000 transactions (iteration 1,000,000)
Saving datasets...
Dataset generation complete!
- Sessions: 1,000,000 (target: 1,000,000)
- Transactions: 250,000 (target: 250,000)
- Remaining products: 1,515,702
```

Screenshot 1 - Dataset generation (dataset_generator.py at SCALE=0.5)

Windows PowerShell - 00b_docker

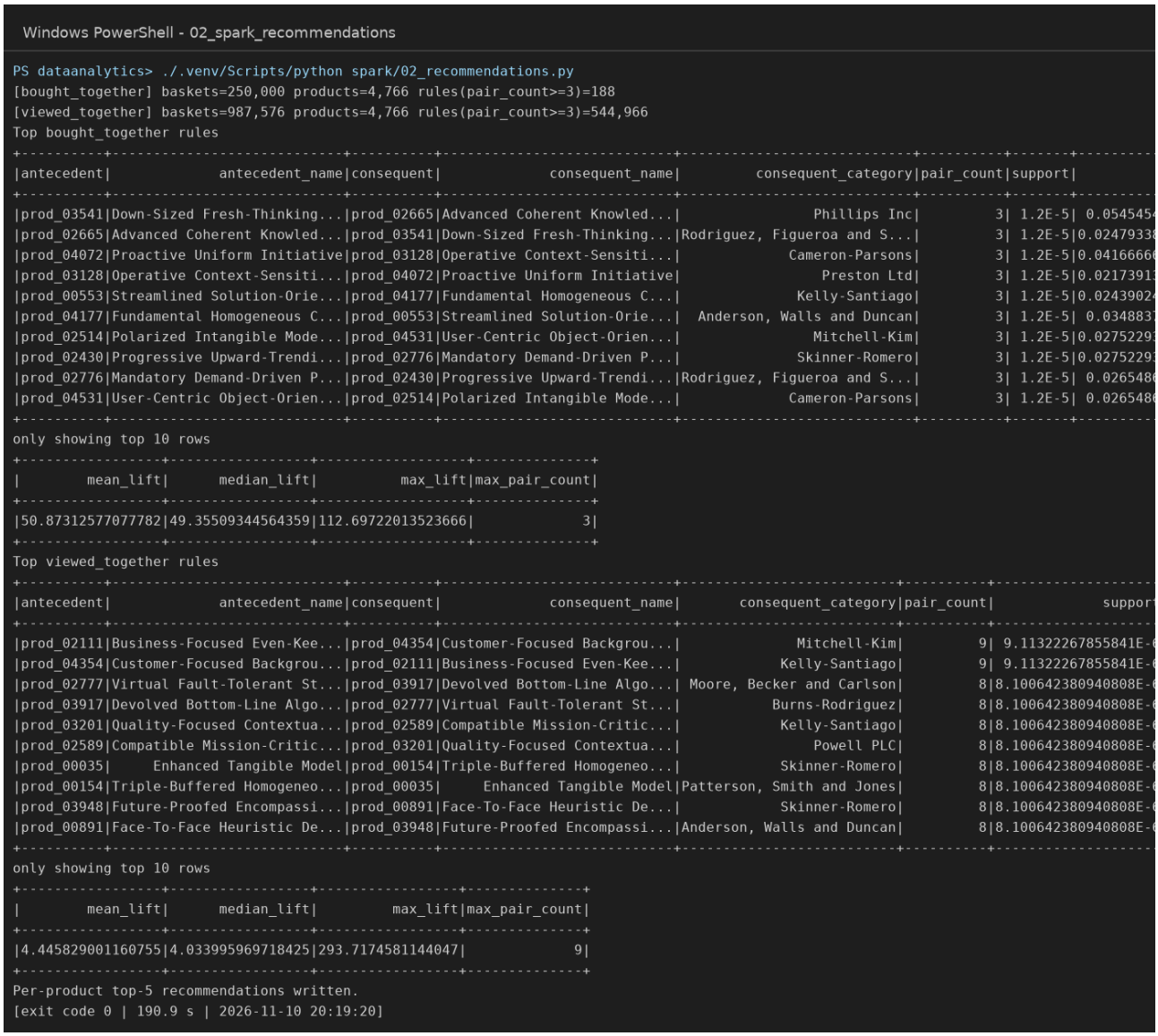
```
PS dataanalytics> docker ps --format table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}
NAMES        IMAGE                  STATUS          PORTS
auca-mongo   mongo:7.0              Up 7 minutes    0.0.0.0:27018->27017/tcp, [::]:27018->27017/tcp
auca-hbase   harisekhon/hbase:2.1   Up 13 minutes   0.0.0.0:2181->2181/tcp, [::]:2181->2181/tcp, 0.0.0.0:9090->9090/tcp, [::]:9090->9090/tc
[exit code 0 | 0.2 s | 2026-11-10 20:29:49]
```

Screenshot 2 - Docker containers: MongoDB 7.0 (host port 27018) and HBase 2.1 (Thrift 9090, UI 16010)

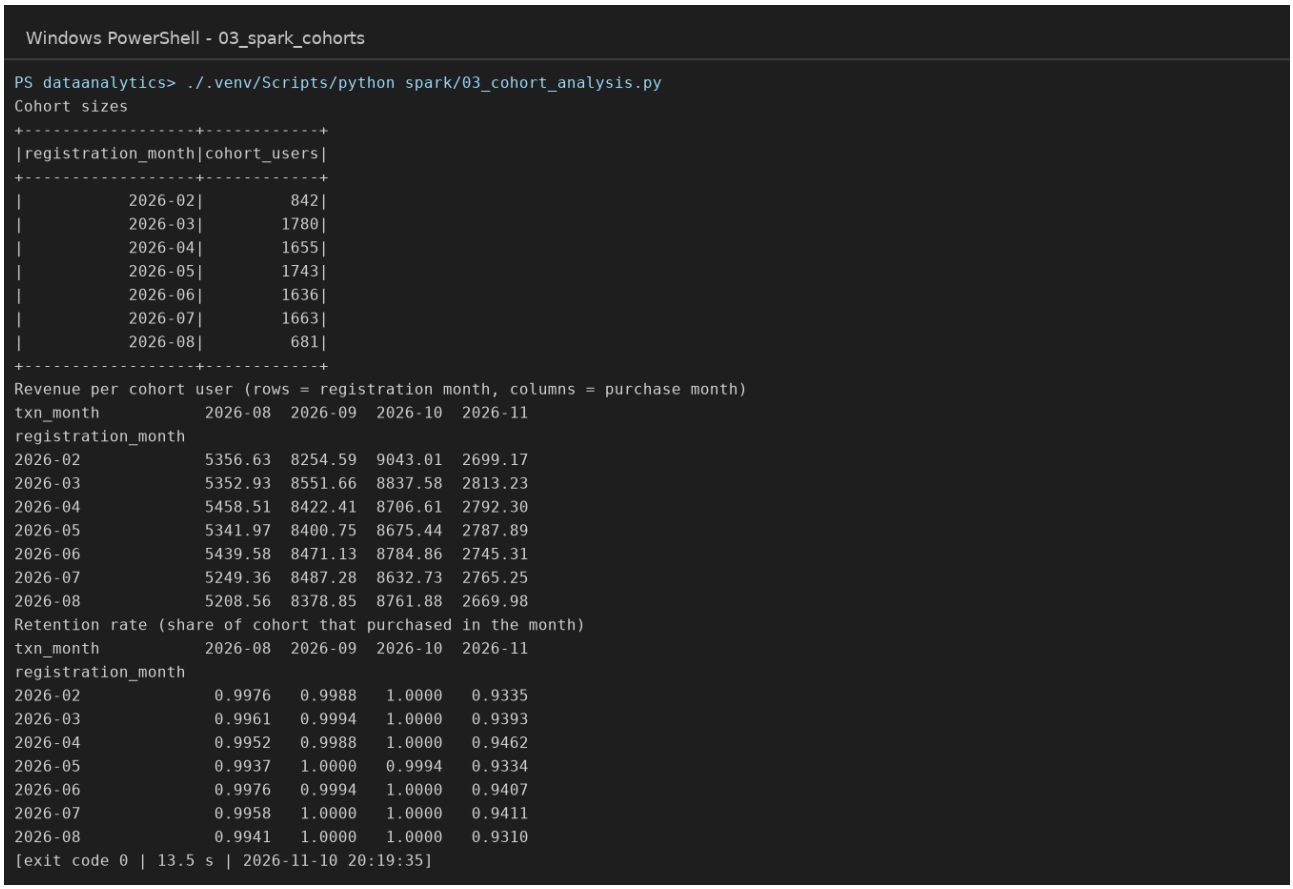
Windows PowerShell - 01_spark_clean

```
PS dataanalytics> ./.venv/Scripts/python spark/01_clean_normalize.py
Cleaning raw dataset from C:/aucadata\raw
  wrote users                        10,000 rows
  wrote categories                       25 rows
  wrote subcategories                   109 rows
  wrote products                      5,000 rows
  wrote product_price_history         9,962 rows
  wrote page_views                9,993,670 rows
  wrote cart_items                1,036,150 rows
  wrote sessions                    999,999 rows
  wrote transaction_items           516,977 rows
  wrote transactions                250,000 rows
Transactions with subtotal - discount != total: 0
Done.
[exit code 0 | 243.5 s | 2026-11-10 20:16:08]
```

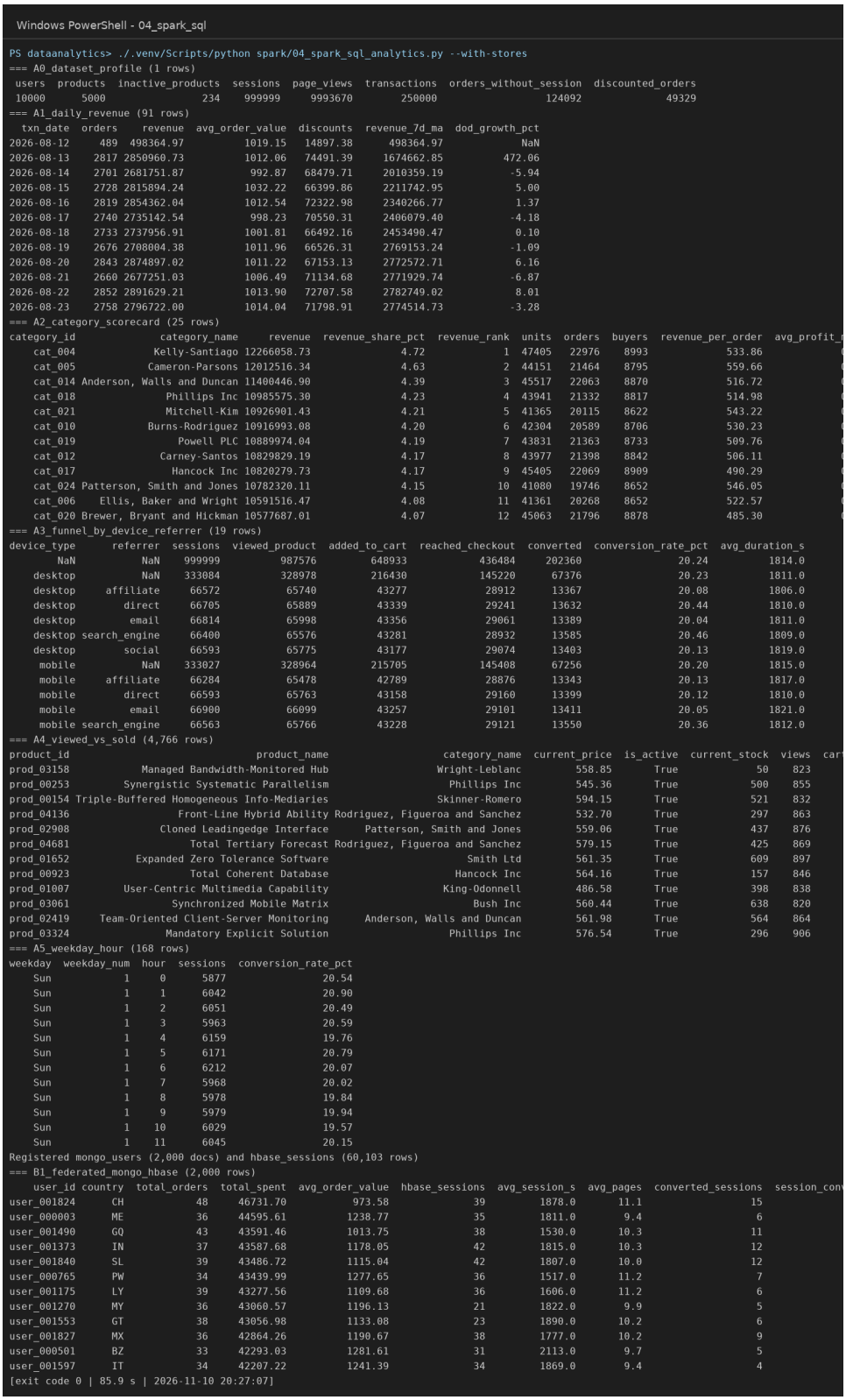
Screenshot 3 - Spark cleaning / normalisation job writing the curated Parquet tables



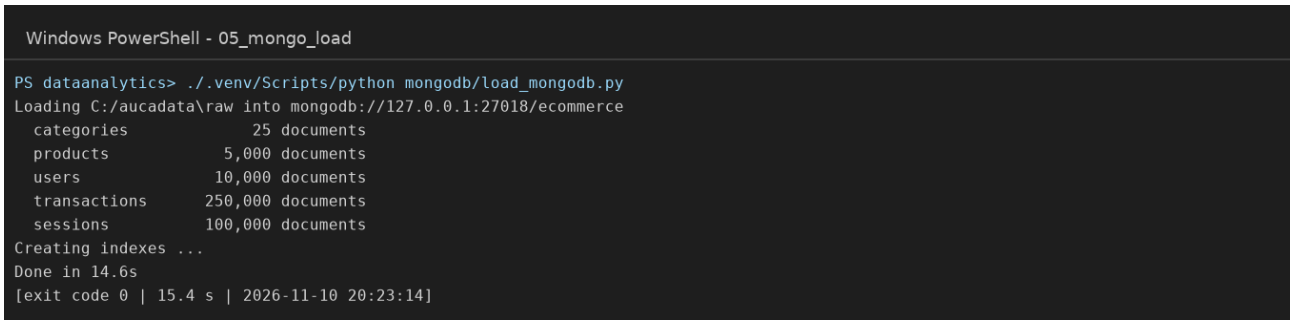
Screenshot 4 - Spark market-basket recommendations



Screenshot 5 - Spark cohort analysis



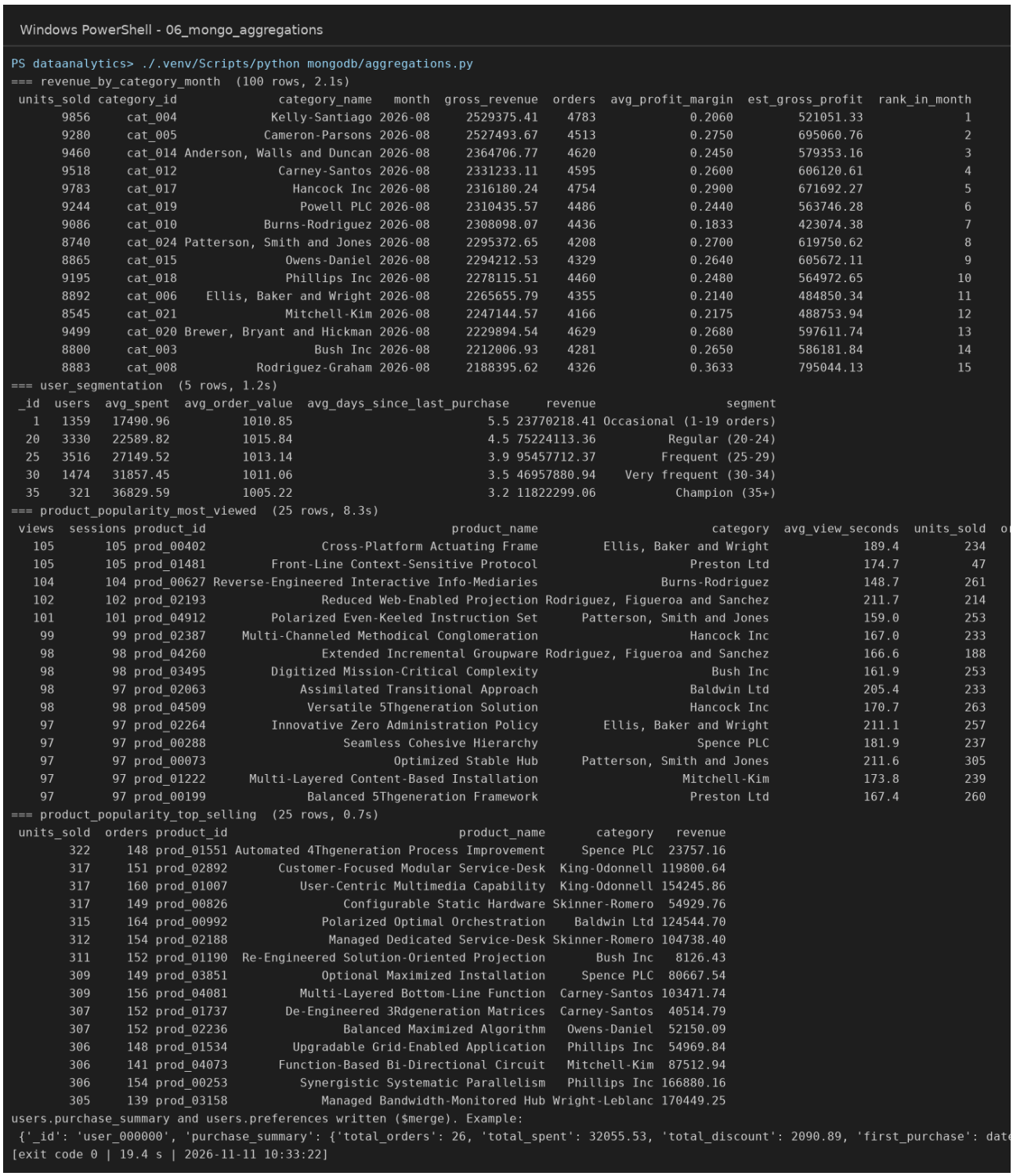
Screenshot 6 - Spark SQL analytics (Section A) and federated MongoDB + HBase query (Section B)



```
Windows PowerShell - 05_mongo_load

PS dataanalytics> ./.venv/Scripts/python mongodb/load_mongodb.py
Loading C:/aucadata\raw into mongodb://127.0.0.1:27018/ecommerce
  categories             25 documents
  products            5,000 documents
  users              10,000 documents
  transactions      250,000 documents
  sessions          100,000 documents
Creating indexes ...
Done in 14.6s
[exit code 0 | 15.4 s | 2026-11-10 20:23:14]
```

Screenshot 7 - MongoDB loader: collections and indexes



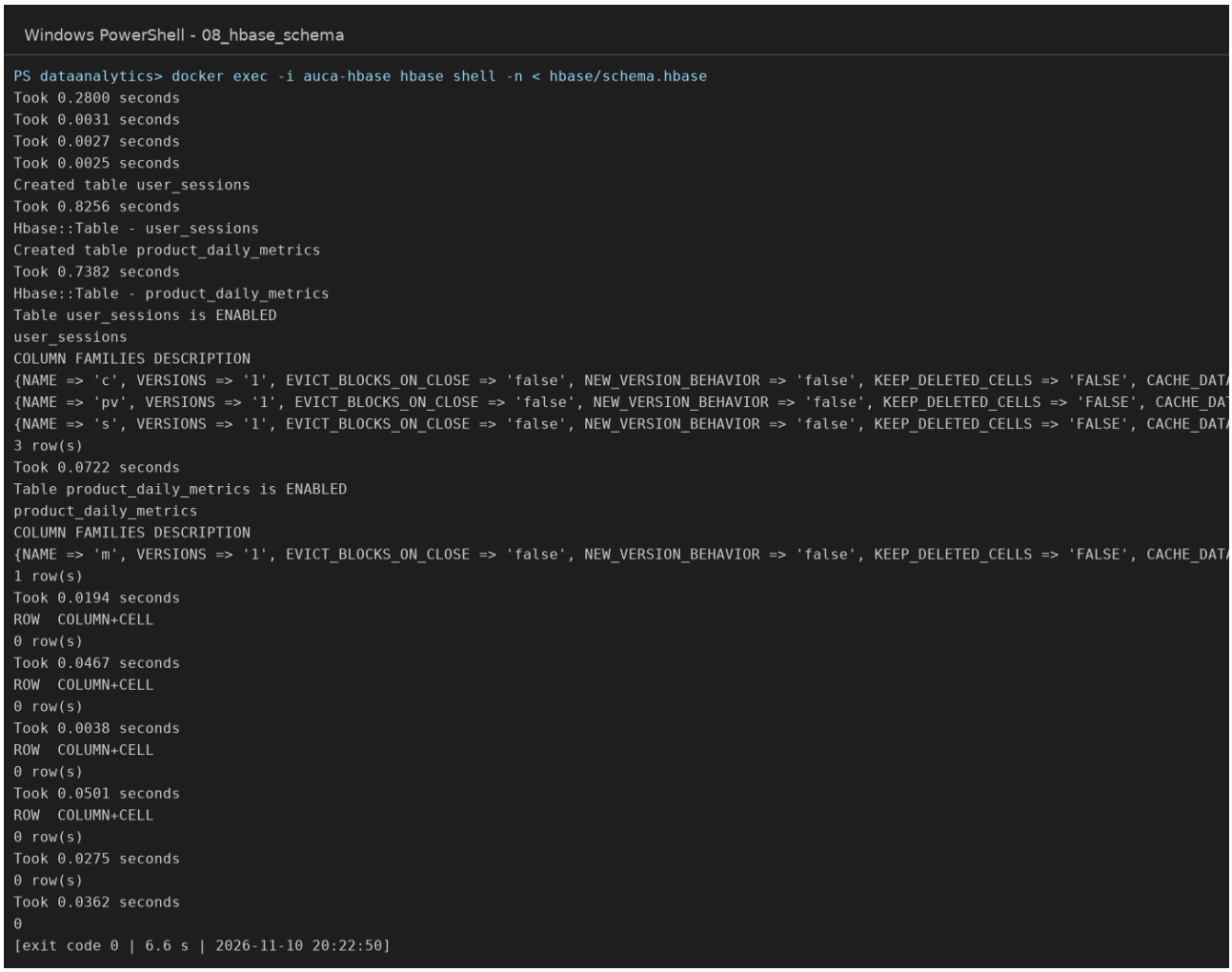
```
Windows PowerShell - 06_mongo_aggregations

PS dataanalytics> ./.venv/Scripts/python mongodb/aggregations.py
=== revenue_by_category_month  (100 rows, 2.1s)
 units_sold category_id              category_name   month  gross_revenue  orders  avg_profit_margin  est_gross_profit  rank_in_month
       9856     cat_004             Kelly-Santiago 2026-08     2529375.41    4783             0.2060         521051.33              1
       9280     cat_005           Cameron-Parsons 2026-08     2527493.67    4513             0.2750         695060.76              2
       9460     cat_014 Anderson, Walls and Duncan 2026-08     2364706.77    4620             0.2450         579353.16              3
       9518     cat_012             Carney-Santos 2026-08     2331233.11    4595             0.2600         606120.61              4
       9783     cat_017               Hancock Inc 2026-08     2316180.24    4754             0.2900         671692.27              5
       9244     cat_019                Powell PLC 2026-08     2310435.57    4486             0.2440         563746.28              6
       9086     cat_010           Burns-Rodriguez 2026-08     2308098.07    4436             0.1833         423074.38              7
       8740     cat_024 Patterson, Smith and Jones 2026-08     2295372.65    4208             0.2700         619750.62              8
       8865     cat_015              Owens-Daniel 2026-08     2294212.53    4329             0.2640         605672.11              9
       9195     cat_018               Phillips Inc 2026-08     2278115.51    4460             0.2480         564972.65             10
       8892     cat_006    Ellis, Baker and Wright 2026-08     2265655.79    4355             0.2140         484850.34             11
       8545     cat_021              Mitchell-Kim 2026-08     2247144.57    4166             0.2175         488753.94             12
       9499     cat_020 Brewer, Bryant and Hickman 2026-08     2229894.54    4629             0.2680         597611.74             13
       8800     cat_003                  Bush Inc 2026-08     2212006.93    4281             0.2650         586181.84             14
       8883     cat_008          Rodriguez-Graham 2026-08     2188395.62    4326             0.3633         795044.13             15
=== user_segmentation  (5 rows, 1.2s)
 _id  users  avg_spent  avg_order_value  avg_days_since_last_purchase     revenue                 segment
   1   1359   17490.96          1010.85                          5.5 23770218.41 Occasional (1-19 orders)
  20   3330   22589.82          1015.84                          4.5 75224113.36          Regular (20-24)
  25   3516   27149.52          1013.14                          3.9 95457712.37         Frequent (25-29)
  30   1474   31857.45          1011.06                          3.5 46957880.94    Very frequent (30-34)
  35    321   36829.59          1005.22                          3.2 11822299.06           Champion (35+)
=== product_popularity_most_viewed  (25 rows, 8.3s)
 views  sessions product_id                                product_name                      category  avg_view_seconds  units_sold  o
   105       105 prod_00402                 Cross-Platform Actuating Frame        Ellis, Baker and Wright             189.4         234
   105       105 prod_01481         Front-Line Context-Sensitive Protocol                    Preston Ltd             174.7          47
   104       104 prod_00627 Reverse-Engineered Interactive Info-Mediaries                Burns-Rodriguez             148.7         261
   102       102 prod_02193               Reduced Web-Enabled Projection Rodriguez, Figueroa and Sanchez             211.7         214
   101       101 prod_04912         Polarized Even-Keeled Instruction Set      Patterson, Smith and Jones             159.0         253
    99        99 prod_02387      Multi-Channeled Methodical Conglomeration                  Hancock Inc             167.0         233
    98        98 prod_04260               Extended Incremental Groupware Rodriguez, Figueroa and Sanchez             166.6         188
    98        98 prod_03495         Digitized Mission-Critical Complexity                       Bush Inc             161.9         253
    98        97 prod_02063             Assimilated Transitional Approach                    Baldwin Ltd             205.4         233
    98        98 prod_04509              Versatile 5Thgeneration Solution                    Hancock Inc             170.7         263
    97        97 prod_02264         Innovative Zero Administration Policy        Ellis, Baker and Wright             211.1         257
    97        97 prod_00288                   Seamless Cohesive Hierarchy                     Spence PLC             181.9         237
    97        97 prod_00073                        Optimized Stable Hub      Patterson, Smith and Jones             211.6         305
    97        97 prod_01222      Multi-Layered Content-Based Installation                    Mitchell-Kim             173.8         239
    97        97 prod_00199               Balanced 5Thgeneration Framework                    Preston Ltd             167.4         260
=== product_popularity_top_selling  (25 rows, 0.7s)
 units_sold  orders product_id                                product_name       category   revenue
        322     148 prod_01551 Automated 4Thgeneration Process Improvement     Spence PLC  23757.16
        317     151 prod_02892       Customer-Focused Modular Service-Desk  King-Odonnell 119800.64
        317     160 prod_01007          User-Centric Multimedia Capability  King-Odonnell 154245.86
        317     149 prod_00826                Configurable Static Hardware Skinner-Romero  54929.76
        315     164 prod_00992             Polarized Optimal Orchestration    Baldwin Ltd 124544.70
        312     154 prod_02188              Managed Dedicated Service-Desk Skinner-Romero 104738.40
        311     152 prod_01190  Re-Engineered Solution-Oriented Projection       Bush Inc   8126.43
        309     149 prod_03851             Optional Maximized Installation     Spence PLC  80667.54
        309     156 prod_04081          Multi-Layered Bottom-Line Function  Carney-Santos 103471.74
        307     152 prod_01737        De-Engineered 3Rdgeneration Matrices  Carney-Santos  40514.79
        307     152 prod_02236                Balanced Maximized Algorithm   Owens-Daniel  52150.09
        306     148 prod_01534         Upgradable Grid-Enabled Application   Phillips Inc  54969.84
        306     141 prod_04073       Function-Based Bi-Directional Circuit   Mitchell-Kim  87512.94
        306     154 prod_00253          Synergistic Systematic Parallelism   Phillips Inc 166880.16
        305     139 prod_03158             Managed Bandwidth-Monitored Hub Wright-Leblanc 170449.25
users.purchase_summary and users.preferences written ($merge). Example:
 {'_id': 'user_000000', 'purchase_summary': {'total_orders': 26, 'total_spent': 32055.53, 'total_discount': 2090.89, 'first_purchase': date
[exit code 0 | 19.4 s | 2026-11-11 10:33:22]
```

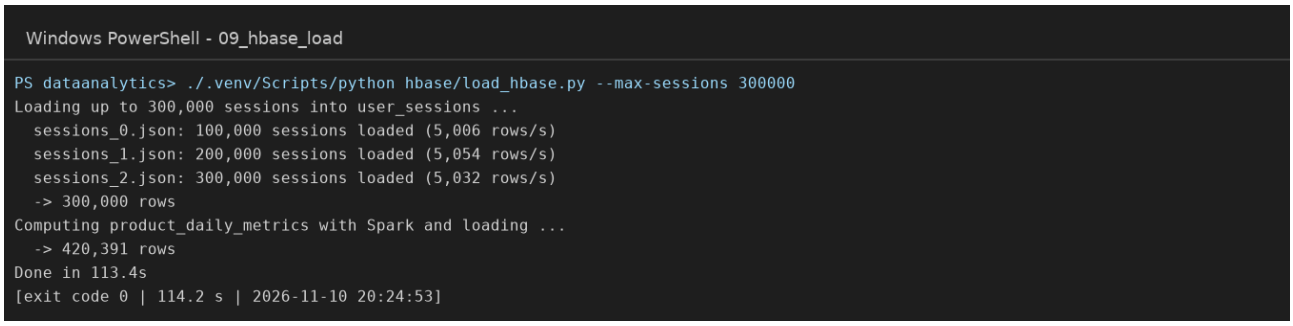
Screenshot 8 - MongoDB aggregation pipelines



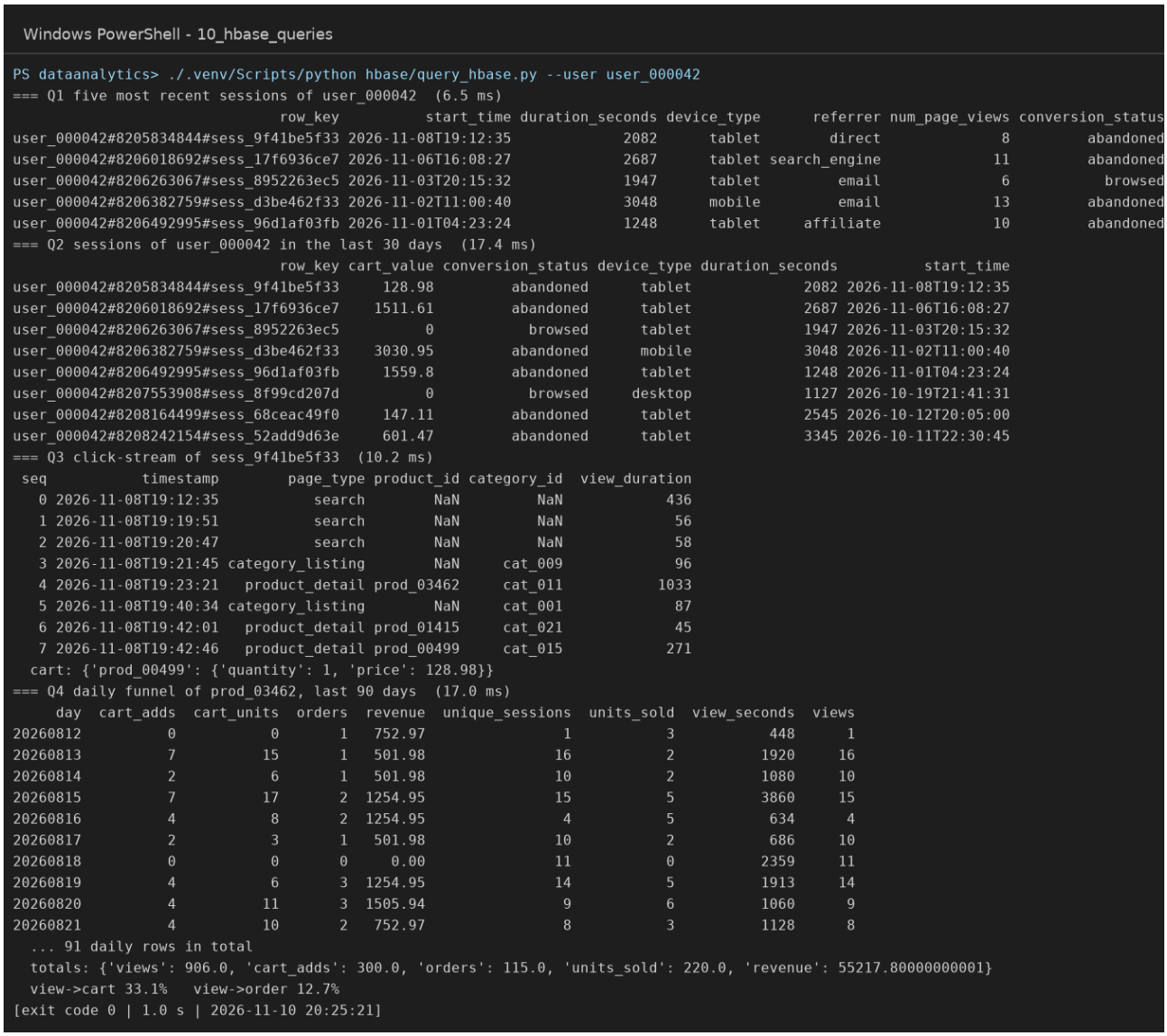
Screenshot 9 - MongoDB shell: user document with the $merge'd purchase_summary



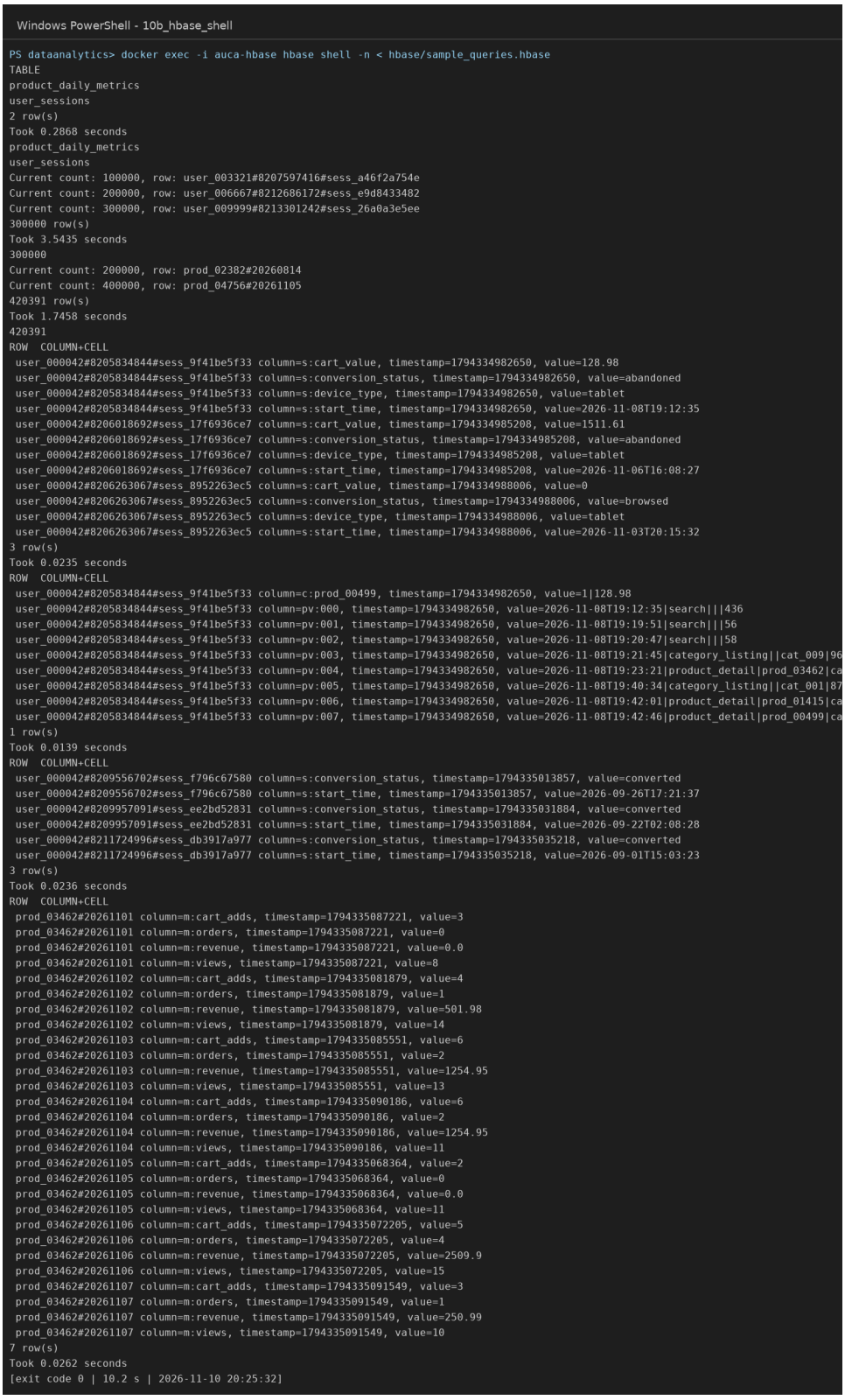
Screenshot 10 - HBase shell: table creation (schema.hbase) and sample scans



Screenshot 11 - HBase loader (happybase / Thrift)



Screenshot 12 - HBase queries: recent sessions, time window, click-stream, product funnel



Screenshot 13 - HBase shell: prefix scans, sparse page-view and cart columns, server-side filter, product date-range scan

APACHE HBASE | Home | Table Details | Procedures & Locks | Process Metrics | Local Logs | Log Level | Debug Dump | Metrics Dump | HBase Configuration

Master hbase

Region Servers

Base Stats | Memory | Requests | Storefiles | Compactions | Replications

ServerName	Start time	Last contact	Version	Requests Per Second	Num. Regions
hbase,16020,1794334553345	Tue Nov 10 18:15:53 GMT 2026	1 s	2.1.3	0	12
Total:1				0	12

Backup Masters

ServerName	Port	Start Time
Total:0		

Tables

User Tables | System Tables | Snapshots

2 table(s) in set. [Details]

Namespace	Name	State	Regions								Description
			OPEN	OPENING	CLOSED	CLOSING	OFFLINE	FAILED	SPLIT	Other	
default	product_daily_metrics	ENABLED	5	0	0	0	0	0	0	0	'product_daily_metrics', {NAME => 'm', DATA_BLOCK_ENCODING => 'FAST_DIFF', COMPRESSION => 'GZ'}
default	user_sessions	ENABLED	5	0	0	0	0	0	0	0	'user_sessions', {NAME => 'c', COMPRESSION => 'GZ'}, {NAME => 'pv', DATA_BLOCK_ENCODING => 'FAST_DIFF', COMPRESSION => 'GZ'}, {NAME => 's', DATA_BLOCK_ENCODING => 'FAST_DIFF', COMPRESSION => 'GZ'}

Peers

Peer Id	Cluster Key	Endpoint	State	IsSerial	Bandwidth	ReplicateAll	Namespaces	Exclude Namespaces	Table Cfs	Exclude Table Cfs
Total: 0										

Tasks

Show All Monitored Tasks | Show non-RPC Tasks | Show All RPC Handler Tasks | Show Active RPC Calls | Show Client Operations

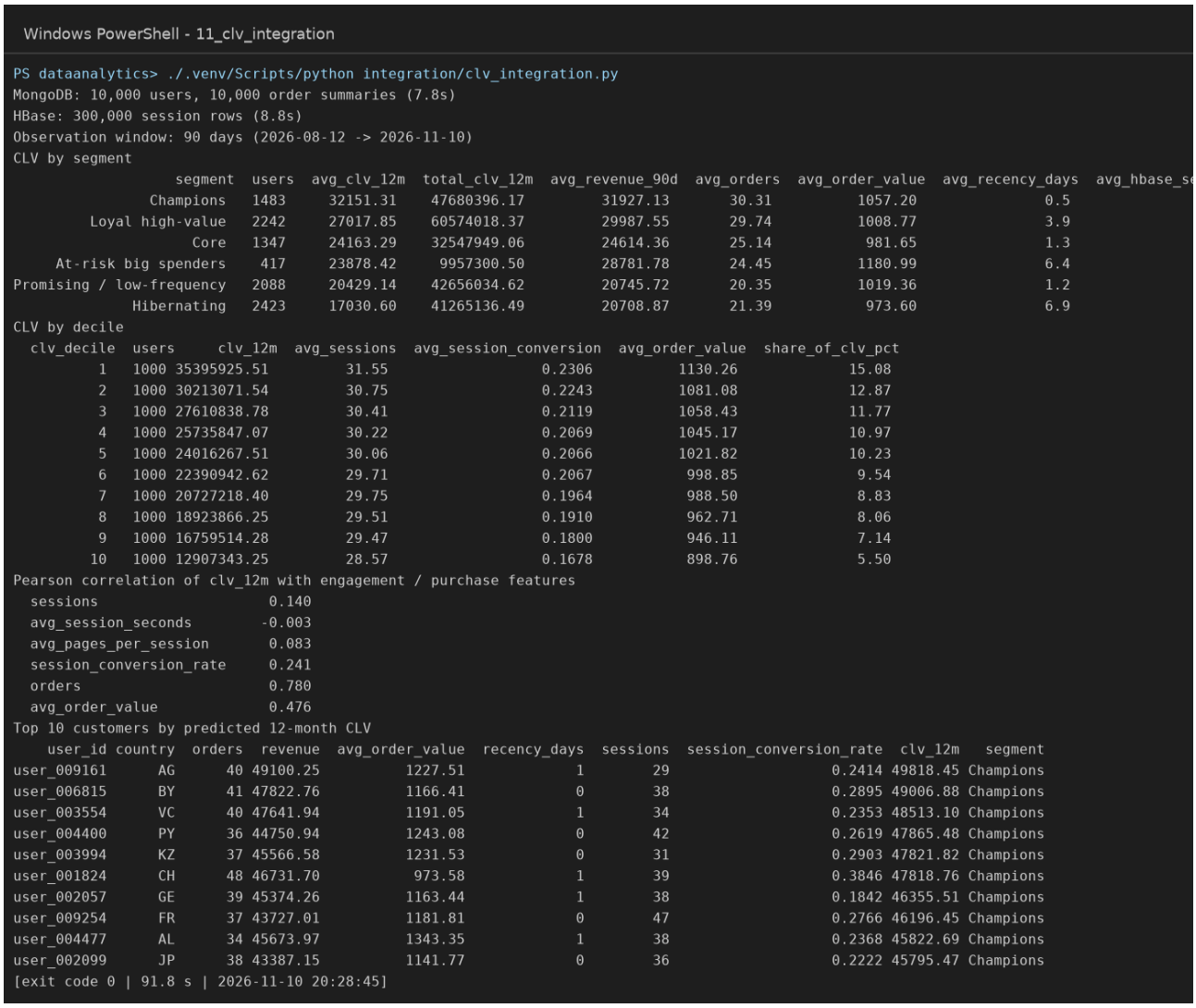
View as JSON

No tasks currently running on this node.

Software Attributes

Attribute Name	Value	Description
HBase Version	2.1.3, revision=da5ec9e4c06c537213883cca8f3cc9a7c19daf67	HBase version and revision

Screenshot 14 - HBase Master web UI (localhost:16010): both tables enabled with 5 regions each (pre-split) and their column-family settings



Screenshot 15 - Integrated CLV query (MongoDB + HBase + Spark)